Cloudflare CDN — Learning Document

> **Level:** Intermediate | **Goal:** Understand how Cloudflare CDN works and how to set it up

---

Table of Contents

---

1. Introduction to CDN

What is a CDN?

A **Content Delivery Network (CDN)** is a globally distributed network of servers that delivers web content to users from the server geographically closest to them — rather than always fetching from the origin server.

Without a CDN, every user request travels all the way to your origin server (e.g., a data center in the US), regardless of where the user is located. This increases **latency**, especially for users far away.

Why CDNs Exist

Problem	CDN Solution
High latency for distant users	Serve content from nearby edge servers
Origin server overload	Cache content and reduce origin requests
Slow static asset delivery	Serve CSS, JS, images from edge cache
Vulnerability to traffic spikes	Distribute load across many nodes

How Content Delivery Works

1. User types `example.com` in their browser.

2. DNS resolves the domain to the CDN's edge server IP (closest to the user).
3. The edge server checks its **cache** for the requested content.
    - **Cache HIT** → Content is served directly from the edge. Fast! ⚡
    - **Cache MISS** → Edge fetches from the origin server, caches it, then serves the user.
4. Subsequent requests for the same content are served from cache until the **TTL (Time to Live)** expires.

---

2. How Cloudflare CDN Works

Cloudflare's Global Network

Cloudflare operates one of the largest networks in the world, with **300+ Points of Presence (PoPs)** across 100+ countries. Each PoP contains edge servers that can cache content and process requests locally.

Anycast Routing

Cloudflare uses **Anycast** — a routing technique where multiple servers share the same IP address. When a user makes a request, the network automatically routes it to the **nearest and healthiest** edge node.

```
User (India) ──→ Anycast IP ──→ Mumbai PoP   ✅ (closest)
User (UK)    ──→ Anycast IP ──→ London PoP   ✅ (closest)
User (USA)   ──→ Anycast IP ──→ NYC PoP      ✅ (closest)
```

This means users around the world get low-latency responses without you doing anything special.

Request Lifecycle on Cloudflare

```
Browser
  |
  ▼
DNS Lookup → Cloudflare Nameservers return edge IP
  |
  ▼
Cloudflare Edge Server (nearest PoP)
  |
  ├── Cache HIT?  → Return cached response (fast)
  |
```

```
└── Cache MISS? → Forward to Origin Server
                    |
                    ▼
               Origin responds → Edge caches it → Returns to user
```

Cache-Control and TTL

Cloudflare respects HTTP caching headers sent by your origin:

- `Cache-Control: max-age=86400` → Cache for 24 hours
- `Cache-Control: no-store` → Never cache this response
- `ETag` / `Last-Modified` → Used for cache validation (revalidation requests)

You can also override these settings directly in the Cloudflare dashboard using **Cache Rules**.

---

3. Cloudflare-Specific Features

3.1 Caching

- **Default Caching:** Cloudflare automatically caches static assets (images, CSS, JS, fonts) based on file extension.
- **Cache Rules:** Custom rules to cache (or bypass) specific URLs, paths, or request types.
- **Cache Purging:** Instantly invalidate cached content via dashboard or API when you deploy updates.

3.2 Tiered Cache

Cloudflare's **Tiered Cache** adds an intermediate caching layer between the edge and origin. Instead of every edge PoP going back to your origin on a cache miss, they first check an **upper-tier PoP** (regional hub).

```
Edge PoP (Chennai) ──MISS──> Upper Tier (Mumbai) ──MISS──> Origin
                                └──HIT──> Serve from regional cache
```

This reduces origin load significantly.

3.3 Argo Smart Routing *(paid)*

Routes requests through Cloudflare's private backbone instead of the public internet, finding the **fastest real-time path** to your origin. Reduces latency by 30%+ in many cases.

3.4 Cloudflare Workers

Workers allow you to run JavaScript (or WASM) at the edge — on Cloudflare's servers — before requests hit your origin. Use cases include:

- A/B testing at the edge
- Custom authentication
- Modifying request/response headers
- Serving dynamic content without an origin

3.5 DDoS Protection

Cloudflare sits in front of your origin and absorbs **DDoS attacks** (volumetric, protocol, and application layer) automatically. Your origin IP stays hidden from the internet.

3.6 SSL/TLS

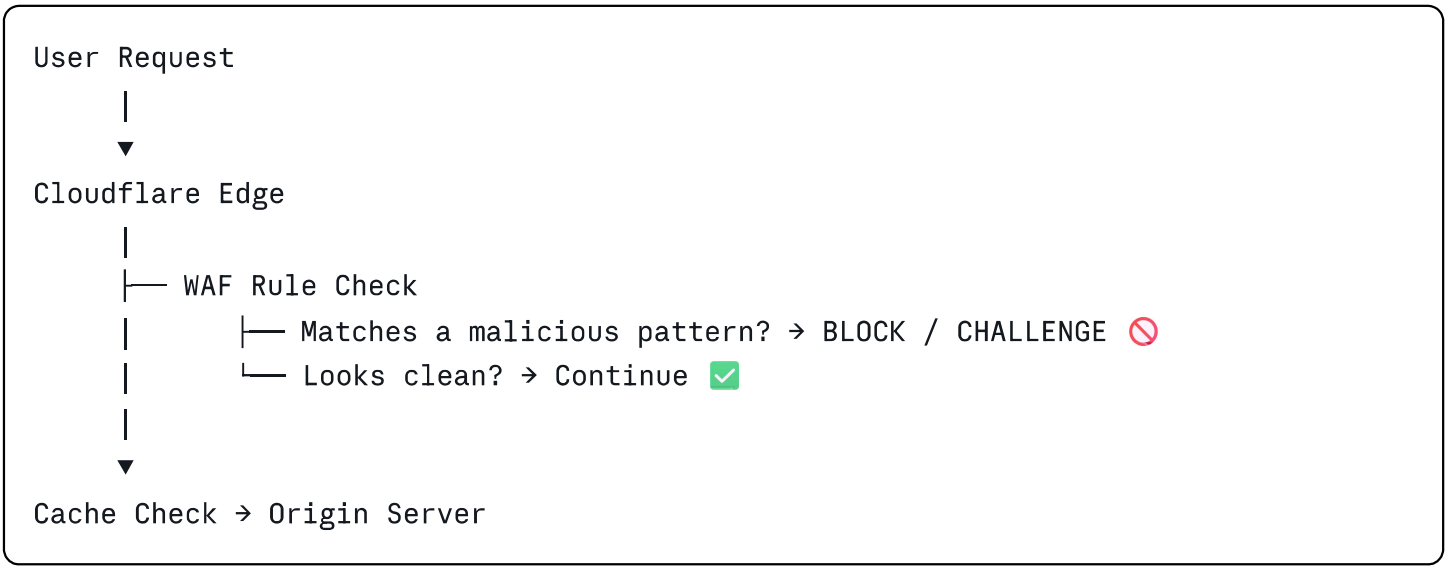
Cloudflare provides **free SSL certificates** and terminates TLS at the edge. Encryption modes:

Mode	Description
Flexible	HTTPS between user ↔ Cloudflare, HTTP to origin
Full	HTTPS end-to-end, but origin cert can be self-signed
Full (Strict)	HTTPS end-to-end with a valid cert on origin

3.7 Web Application Firewall (WAF)

Cloudflare's WAF inspects **incoming HTTP/HTTPS requests** at the edge and blocks malicious traffic before it ever reaches your origin server. It operates at **Layer 7 (Application Layer)** of the OSI model.

How WAF Works



Types of WAF Rules

Rule Type	Description
Managed Rules	Pre-built rules by Cloudflare covering OWASP Top 10 (SQLi, XSS, RCE, etc.)
Custom Rules	Rules you write based on IP, country, URI, headers, user-agent, etc.
Rate Limiting Rules	Block IPs that send too many requests in a time window
Bot Fight Mode	Detects and challenges automated bot traffic

OWASP Top 10 Threats Blocked by WAF

Cloudflare's managed ruleset covers the most critical web vulnerabilities:

- **SQL Injection (SQLi)** — e.g., `' OR 1=1 --` in form inputs
- **Cross-Site Scripting (XSS)** — injecting malicious scripts into web pages
- **Remote Code Execution (RCE)** — attempts to run commands on your server
- **Path Traversal** — e.g., `../../etc/passwd` in URL parameters
- **Log4Shell / known CVEs** — Cloudflare pushes rule updates for zero-days quickly

WAF Actions

When a rule matches, Cloudflare can take one of these actions:

Action	Behavior
Block	Request is dropped, user gets a 403 error
Challenge (CAPTCHA)	User must solve a CAPTCHA to proceed
JS Challenge	Silent JavaScript check to verify real browser
Managed Challenge	Cloudflare decides challenge type automatically
Log	Allow the request but log it for analysis
Skip	Bypass other rules (used for trusted IPs)

Custom WAF Rule Example

Block all requests from a specific country to your admin panel:

```
(http.request.uri.path contains "/admin" and ip.geoip.country eq "XX")
→ Action: Block
```

Block requests with suspicious SQL patterns in query strings:

```
(http.request.uri.query contains "UNION SELECT" or
 http.request.uri.query contains "' OR '1'='1")
→ Action: Block
```

WAF vs DDoS Protection

These are complementary, not the same:

Feature	Protects Against	Layer
WAF	Application-level attacks (SQLi, XSS, bots)	Layer 7
DDoS Protection	Volumetric floods, protocol attacks	Layer 3/4/7

> 💡 **Free plan** includes the Cloudflare Managed Ruleset (limited). **Pro plan and above** unlock the full OWASP ruleset and custom rules.

3.8 Cloudflare Dashboard Overview

Key sections in the dashboard:

- **DNS** — Manage DNS records (proxied vs DNS-only)
- **SSL/TLS** — Configure encryption mode
- **Caching** — Cache rules, purge cache, tiered cache settings
- **Speed** — Auto Minify, image optimization, Rocket Loader
- **Security** — WAF, DDoS settings, bot management
- **Workers & Pages** — Edge functions and static site hosting

---

4. Setup & Configuration Steps

Step 1: Create a Cloudflare Account

1. Go to cloudflare.com and sign up.
2. Click **"Add a Site"** and enter your domain (e.g., `example.com`).
3. Choose a plan (Free plan works for most learning/personal use).

Step 2: Cloudflare Scans Your DNS

Cloudflare automatically scans and imports your existing DNS records. Review them carefully before proceeding.

Step 3: Update Your Nameservers

Cloudflare will give you two custom nameservers, e.g.:

```
ada.ns.cloudflare.com
bob.ns.cloudflare.com
```

Go to your domain registrar (GoDaddy, Namecheap, etc.) and replace the existing nameservers with these. DNS propagation can take **up to 48 hours**, but usually takes minutes.

Step 4: Configure DNS Records

In the Cloudflare DNS tab, ensure your records are set correctly:

Type	Name	Value	Proxied?
A	`@`	`your-origin-IP`	✅ Yes (orange cloud)
CNAME	`www`	`example.com`	✅ Yes
MX	`@`	`mail.example.com`	❌ No (DNS only)

> **Orange cloud (Proxied)** = Traffic goes through Cloudflare CDN
> **Grey cloud (DNS only)** = Cloudflare just resolves DNS, no CDN benefits

Step 5: Set SSL Mode

Go to **SSL/TLS → Overview** and set the encryption mode. For most setups, use **Full (Strict)** if your origin has a valid SSL cert.

Step 6: Configure Caching

Go to **Caching → Configuration**:

- Set **Browser Cache TTL** (how long browsers cache content)
- Enable **Always Online** (serves cached pages if origin is down)
- Create **Cache Rules** for specific paths if needed

Step 7: Test Your Setup

Check if CDN is working using response headers:

```bash
curl -I https://example.com/image.png
```

Look for:

```
CF-Cache-Status: HIT       # Served from Cloudflare cache ✅
CF-Cache-Status: MISS      # Fetched from origin (first request)
CF-Cache-Status: BYPASS    # Caching was bypassed
CF-RAY: 7a1b2c3d4e5f-BOM   # Cloudflare Ray ID (BOM = Mumbai PoP)
```

---

5. Use Cases & Examples

Use Case 1: Static Website Acceleration

A portfolio or blog with HTML, CSS, JS, and images benefits immediately from Cloudflare CDN. Static assets are cached at edge nodes globally, so users in any country load your site quickly.

Result: Page load time reduced from ~800ms to ~120ms for users far from the origin.

Use Case 2: API Response Caching

Cache GET responses from your REST API at the edge for a short TTL (e.g., 60 seconds). This dramatically reduces database load for high-traffic endpoints.

```
Cache-Control: public, max-age=60, s-maxage=60
```

> ⚠️ Never cache authenticated or user-specific API responses.

Use Case 3: Media & Image Delivery

Serving large images or video thumbnails? Cloudflare caches them at edge nodes so the same asset isn't repeatedly fetched from your origin (e.g., S3 bucket or server).

Combine with **Cloudflare Images** or **Polish** (image compression) for further optimization.

Use Case 4: Reducing Origin Load During Traffic Spikes

A product launch or viral post can spike traffic 100x. With Cloudflare's CDN caching your homepage and assets, the origin only handles a fraction of requests — mostly cache misses and dynamic content.

Real-World Example Walkthrough

Scenario: You run a news website hosted on a VPS in Frankfurt.

1. A user in Mumbai visits `news-site.com/article/123`.
2. DNS resolves via Cloudflare → routes to the **Mumbai PoP**.
3. First visit: Cache MISS → Cloudflare fetches from Frankfurt origin, caches the article HTML for 5 minutes.
4. Next 1,000 users in India visiting the same article → all served from Mumbai cache. **Origin gets 0 requests.**
5. After 5 minutes, TTL expires → next request triggers a fresh fetch from origin.

Key Takeaways

- A CDN reduces latency by serving content from servers close to the user.
- Cloudflare uses **Anycast routing** to direct users to the nearest PoP automatically.
- The **orange cloud** in Cloudflare DNS means traffic is proxied through the CDN.
- Always check `CF-Cache-Status` headers to verify caching behavior.
- Cloudflare's free plan covers CDN, DDoS protection, and SSL for most use cases.

Further Reading

- Cloudflare Learning Center
- Cloudflare Docs — Caching
- Cloudflare Docs — Workers
- HTTP Caching — MDN Web Docs